

ORIOS BANK

Project Structure & Architecture Guide

A Comprehensive Deep Dive into the Modern Digital Banking Solution

DEVELOPER

Mark Jeferson D. Tumolva

Java 17+ | Spring Boot 3.2 | MongoDB | JWT Security | HTML5/CSS3/JS | Python 3.x

Table of Contents

- **Project Overview**
1—> What is Orios Bank and its core purpose
- **Complete Directory Structure**
2—> Visual tree with all folders and files
- **Backend Architecture**
3—> Spring Boot layers, models, and services
- **Frontend Architecture**
4—> HTML, CSS, and JavaScript organization
- **CLI Tool**
5—> Python command-line interface
- **API Endpoints Reference**
6—> Complete REST API documentation
- **Database Schema**
7—> MongoDB collections and relationships
- **Authentication & Security**
8—> JWT implementation details
- **Feature Explanations**
9—> How each banking feature works
- **Deployment & Setup**
10—> Step-by-step installation guide

2. Project Overview

Orios Bank is a modern, full-stack digital banking application designed to simulate real-world banking operations in a secure, responsive web environment. It demonstrates enterprise-grade software architecture patterns using Java Spring Boot for the backend, vanilla JavaScript for the frontend, and MongoDB as the NoSQL database.

Core Philosophy

The project follows a **clean architecture** approach where concerns are strictly separated:

- Backend handles all business logic, security, and data persistence
- Frontend provides an interactive, animated user experience
- CLI offers programmatic access for automation and scripting
- Database stores all entities in flexible document collections

Key Capabilities

- **Multi-Account Management:** Users can open multiple Checking and Savings accounts with real-time balance tracking.
- **Secure Transactions:** Deposit, withdraw, and transfer funds with full audit trails and transaction history.
- **Loan System:** Request loans with interest rates, track repayment schedules, and enable automatic debt collection.
- **Credit Cards:** Issue branded VISA and MASTER cards with unique card numbers and privacy controls.
- **Admin Dashboard:** Administrators get system-wide oversight with customer management and loan approval workflows.

2. Complete Directory Structure

The project is organized into four main directories. Each serves a distinct purpose and follows industry-standard naming conventions. Understanding this layout is essential for navigation, debugging, and contributing to the codebase.

Path	Technology	Purpose
oriosbank-refactored/	—	Root project directory
backend/	Java Spring Boot	REST API server & business logic
src/main/java/com/oriosbank/api/	—	Java source code root
controller/	@RestController	HTTP request handlers (API entry points)
service/	@Service	Business logic & transaction processing
model/	@Document	MongoDB entity definitions (POJOs)
repository/	MongoRepository	Data access layer (CRUD operations)
dto/	POJO	Data Transfer Objects for API responses
exception/	@ControllerAdvice	Global error handling & custom exceptions
security/	JWT Filter	Authentication & authorization configuration
src/main/resources/static/	HTML/CSS/JS	Frontend assets served by Spring Boot
pom.xml	Maven	Dependency management & build configuration
frontend/	HTML/CSS/JS	Standalone frontend source (mirrors static/)
index.html	HTML5	Main application page
style.css	CSS3	Light-mode styling with animations
app.js	ES6+	Client-side application logic
cli/	Python 3.x	Command-line interface scripts
orios.py	Python	Interactive menu & argument-based CLI
README.md	Markdown	Project documentation

!Important: The backend/src/main/resources/static/ folder and frontend/ folder contain identical files. Spring Boot automatically serves files from static/ at the root URL. The separate frontend/ directory exists for development convenience and version control clarity.

3. Backend Architecture (Spring Boot)

The backend is built on **Spring Boot 3.2** with Java 17+, following the classic layered architecture pattern. Each layer has a single responsibility, making the codebase testable, maintainable, and scalable.

Controller Layer (controller/)

Annotated with `@RestController`, these classes expose HTTP endpoints. They receive JSON requests, delegate to services, and return JSON responses. Each controller maps to a domain (Auth, Accounts, Loans, Admin). They handle input validation, path variables, and request bodies.

Service Layer (service/)

Annotated with `@Service`, this layer contains all business logic. Services orchestrate multiple repository calls, enforce business rules (e.g., 'insufficient funds'), calculate interest, and manage transactions. This is where the 'brain' of the banking operations lives.

Model Layer (model/)

Annotated with `@Document` (Spring Data MongoDB), these POJOs define the structure of data stored in MongoDB. Each model represents a banking entity: Customer, Account, Transaction, Loan, Card. They use annotations like `@Id` and `@DBRef` for relationships.

Repository Layer (repository/)

Interfaces extending `MongoRepository` or `CrudRepository`. Spring Data automatically generates query implementations at runtime. Custom queries can be added using method naming conventions or `@Query` annotations.

DTO Layer (dto/)

Data Transfer Objects are lightweight classes used to shape API responses. They prevent exposing internal model structures directly. DTOs often combine fields from multiple models for dashboard views or detailed breakdowns.

Exception Layer (exception/)

Custom exception classes plus a global exception handler annotated with `@ControllerAdvice`. This centralizes error responses, ensuring clients always receive consistent JSON error objects with appropriate HTTP status codes.

Security Layer (security/)

Configures Spring Security with JWT. Includes a JwtAuthenticationFilter that intercepts requests, validates tokens from the Authorization header, and sets the security context. The SecurityConfig class defines which endpoints are public vs. protected.

Request Flow Example

1. **1.** Client sends POST /api/accounts/transfer with JWT in header
2. **2.** Security Filter validates JWT and extracts user identity
3. **3.** AccountController receives the request and parses the JSON body
4. **4.** AccountService checks balances, validates accounts, creates debit/credit transactions
5. **5.** AccountRepository & TransactionRepository persist changes to MongoDB
6. **6.** Controller returns a success DTO with updated balances
7. **7.** Client receives JSON and updates the UI

4. Frontend Architecture

The frontend is a **Single Page Application (SPA)** built with vanilla JavaScript (no frameworks). It uses a light-mode design with CSS Grid, Flexbox, custom animations, and gradient backgrounds. All three files work together to create a seamless banking experience.

index.html

The entry point and structural skeleton. It contains the login/registration forms, dashboard layout, account cards, transaction tables, loan request forms, and admin panels. Uses semantic HTML5 elements and data attributes for JavaScript hooks. Sections are conditionally shown/hidden based on authentication state and user role (admin vs. customer).

style.css

Comprehensive stylesheet defining the light-mode theme. Key features include: Floating blob animations using CSS keyframes; Glassmorphism cards with backdrop blur; Responsive grid adapting from mobile (1 column) to desktop (3+ columns); Smooth transitions for hover effects, page transitions, and loading states; Privacy toggle CSS classes that mask sensitive numbers (**** *).

app.js

The application engine written in ES6+ JavaScript. Responsibilities: API Client — Fetch wrapper that attaches JWT tokens to every request; State Management — Tracks current user, accounts, transactions in memory; DOM Manipulation — Renders dynamic content (account cards, transaction lists); Event Handling — Form submissions, button clicks, toggle switches; Routing — Shows/hides sections to simulate page navigation; Validation — Client-side form validation before API calls.

Frontend ↔ Backend Communication

The frontend communicates exclusively through the REST API. After login, the JWT token is stored in localStorage. Every subsequent API call includes the header: `Authorization: Bearer <token>`. If the token expires or is invalid, the frontend redirects to the login page. All data displayed in the UI is fetched dynamically — there is no server-side rendering.

5. CLI Tool (Python)

The Command-Line Interface provides an alternative way to interact with the Orios Bank API. Built with Python 3.x using the `requests` library for HTTP and `rich` library for beautiful terminal output, the CLI supports both interactive menus and direct command execution.

Two Modes of Operation

Interactive Mode: Run `python3 orios.py` without arguments to launch a menu-driven interface. The CLI presents numbered options (Login, Register, View Accounts, etc.) and prompts for input. Ideal for exploration and users who prefer guided workflows. The rich library renders colored tables, progress bars, and styled panels.

Argument Mode: Run with command-line arguments for scripting and automation. Examples: `python3 orios.py auth login user@email.com password123`; `python3 orios.py tx transfer ACC001 ACC002 500.0 --desc "Rent"`; `python3 orios.py admin loans approve LOAN123 ACC456`. This mode is perfect for CI/CD pipelines, batch operations, or power users.

CLI Command Categories

- **auth:** login, register — User authentication
- **accounts:** list, open — Account management
- **tx:** deposit, withdraw, transfer — Financial transactions
- **loans:** request, repay — Loan operations
- **admin:** users, accounts, txs, loans, db — Administrative oversight

6. API Endpoints Reference

The REST API follows RESTful conventions with clear HTTP method semantics. All endpoints (except authentication) require a valid JWT token in the Authorization header.

Authentication

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Create new customer account	No
POST	/api/auth/login	Authenticate and receive JWT	No
GET	/api/auth/me	Get current user profile	Yes

Accounts

Method	Endpoint	Description
POST	/api/accounts/open	Open new Checking or Savings account
GET	/api/accounts/my-accounts	List all accounts for logged-in user
GET	/api/accounts/total-balance	Get sum of all account balances
POST	/api/accounts/deposit	Deposit funds into an account
POST	/api/accounts/withdraw	Withdraw funds from an account
POST	/api/accounts/transfer	Transfer between user's accounts
POST	/api/accounts/{id}/toggle-visibility	Show/hide account number
POST	/api/accounts/{id}/issue-card	Issue VISA or MASTER card

Loans

Method	Endpoint	Description	Access
POST	/api/loans/request	Submit a new loan application	Customer
GET	/api/loans/my-loans	View personal loan history	Customer
POST	/api/loans/repay	Make a manual loan repayment	Customer
GET	/api/loans/admin/all	View all system loans	Admin
POST	/api/loans/admin/{id}/approve	Approve a pending loan	Admin
POST	/api/loans/admin/{id}/reject	Reject a pending loan	Admin
POST	/api/loans/admin/{id}/auto-debt	Toggle automatic debt collection	Admin

Administration

Method	Endpoint	Description
GET	/api/accounts/admin/all-customers	List all registered customers
GET	/api/accounts/admin/all-accounts	List all accounts system-wide
GET	/api/transactions/admin/all	View complete transaction history
POST	/api/accounts/admin/accounts/{id}/status	Hold, block, or activate account
DELETE	/api/accounts/admin/accounts/{id}	Delete account (cascading removal)
DELETE	/api/accounts/admin/customers/{id}	Delete customer and all data

7. Database Schema (MongoDB)

Orios Bank uses **MongoDB**, a document-oriented NoSQL database. Unlike relational databases with tables and rows, MongoDB stores data as JSON-like documents in collections. This flexibility is ideal for a banking application where account types and transaction metadata can vary.

Collection: customers

Stores user registration data. Each document contains: `_id` — MongoDB ObjectId (primary key); `fullName` — Customer's display name; `email` — Unique login identifier; `password` — BCrypt-hashed password (never stored plain-text); `phone` — Contact number; `role` — CUSTOMER or ADMIN; `createdAt` — Registration timestamp.

Collection: accounts

Represents bank accounts. Each document contains: `_id` — Unique account identifier (e.g., ACC-2026-XXXX); `customerId` — Reference to the owning customer; `accountType` — CHECKING or SAVINGS; `balance` — Current monetary balance (BigDecimal); `currency` — Default USD; `status` — ACTIVE, HOLD, or BLOCKED; `isVisible` — Privacy toggle for account number display; `cards` — Embedded array of issued credit cards; `createdAt` — Account opening date.

Collection: transactions

Immutable audit trail of all financial movements. Each document contains: `_id` — Unique transaction ID; `fromAccountId` — Source account (null for deposits); `toAccountId` — Destination account (null for withdrawals); `type` — DEPOSIT, WITHDRAWAL, TRANSFER, LOAN_REPAYMENT; `amount` — Transaction amount; `description` — User-provided note; `timestamp` — Exact time of transaction.

Collection: loans

Tracks loan applications and active debts. Each document contains: `_id` — Unique loan identifier; `customerId` — Borrower reference; `amount` — Original principal; `interestRate` — Annual percentage rate; `remainingBalance` — Amount still owed; `status` — PENDING, APPROVED, REJECTED, PAID_OFF; `autoDebtEnabled` — Whether to auto-deduct from account; `repaymentAccountId` — Linked account for auto-payments; `requestedAt` & `approvedAt` — Timeline tracking.

Data Relationships

- One-to-Many: One Customer → Many Accounts, Many Loans, Many Transactions
- Embedded: Cards are embedded within Account documents (they don't exist independently)
- Referenced: Transactions reference Account IDs (to maintain immutable audit history even if accounts are deleted)

8. Authentication & Security

Security is paramount in banking applications. Orios Bank implements a multi-layered security strategy centered around **JWT (JSON Web Tokens)** and Spring Security.

JWT Authentication Flow

1. Registration: User submits email + password. Password is hashed with BCrypt (adaptive one-way hashing) and stored.
2. Login: Server validates credentials and generates a JWT signed with a secret key.
3. Token Structure: The JWT contains three parts — Header (algorithm), Payload (user ID, email, role, expiration), and Signature (verifies integrity).
4. Subsequent Requests: Client sends Authorization: Bearer <token> header.
5. Validation: The `JwtAuthenticationFilter` intercepts every request, parses the token, verifies the signature, checks expiration, and loads the user into Spring's `SecurityContext`.
6. Authorization: Methods annotated with `@PreAuthorize("hasRole('ADMIN')")` reject non-admin tokens with 403 Forbidden.

Password Security

Passwords are never stored in plain text. Spring Security's `BCryptPasswordEncoder` applies the BCrypt algorithm (cost factor 10-12) which automatically salts and hashes passwords. Salting prevents rainbow table attacks, and the adaptive cost factor makes brute-force attacks computationally expensive.

Role-Based Access Control (RBAC)

Two roles exist in the system:

- CUSTOMER — Can manage own accounts, request loans, view personal transactions
- ADMIN — Can view all data, approve loans, delete accounts/customers, manage system status

Endpoints are protected at the controller method level using Spring Security annotations.

Privacy Controls

The frontend implements a visibility toggle that masks sensitive data (account numbers, card numbers, balances) with asterisks. This is a UI-level feature — the backend always returns full data, but the display is controlled by the `isVisible` flag on each account and card.

9. Feature Deep Dives

Account Opening

Customers can open multiple accounts of type CHECKING or SAVINGS. Each account receives a unique generated ID. An optional initial deposit can be made at creation. The account status defaults to ACTIVE and can be changed by admins to HOLD (temporarily frozen) or BLOCKED (permanently restricted).

Fund Transfers

Transfers move money between two accounts belonging to the same customer. The service layer performs the operation atomically: it debits the source account, credits the destination account, and creates two Transaction records (one for each side). If any step fails, the entire operation rolls back to prevent data inconsistency.

Loan Lifecycle

1. Request: Customer submits amount and desired interest rate.
2. Pending: Loan awaits admin review. No funds are disbursed.
3. Approve: Admin approves — funds are deposited to a designated account, remainingBalance is set.
4. Repay: Customer makes manual payments, or auto-debt deducts periodically.
5. Paid Off: When remainingBalance reaches zero, status updates automatically.
6. Reject: Admin denies — loan is archived with REJECTED status.

Credit Card Issuance

When a customer requests a card on an account, the system generates a unique card number, CVV, and expiry date. Two brands are supported: OriosVISA and OriosMASTER. Cards are stored as embedded documents within the Account collection. Each card has its own visibility toggle independent of the account's toggle.

Admin Customer Management

Administrators have a comprehensive dashboard showing:

- Total counts of all entities (customers, accounts, transactions, loans, cards)
- Full customer list with password override capability (for account recovery)
- Account status management (hold/block/activate)
- Cascading deletion — removing a customer automatically removes all their accounts, transactions, and loans to maintain referential integrity.

10. Deployment & Setup Guide

Follow these steps to get Orios Bank running on your local machine. The entire process takes approximately 5-10 minutes depending on your internet connection.

Prerequisites Check

Ensure you have installed:

- Java JDK 17 or higher (java -version)
- Maven 3.8+ (mvn -version)
- MongoDB Community Edition running on localhost:27017 (mongod)
- Python 3.x (optional, for CLI usage)

Step 1: Start MongoDB

Open a terminal and run: `mongod`

Leave this terminal running. MongoDB will listen for connections on the default port 27017.

Step 2: Build the Backend

Navigate to the backend directory and build:

```
cd backend
```

```
mvn clean install
```

This downloads all dependencies (Spring Boot, Spring Security, MongoDB driver, JWT library) and compiles the Java source code. A successful build creates a JAR file in the `target/` directory.

Step 3: Run the Application

Start the Spring Boot server:

```
mvn spring-boot:run
```

The server starts on port 8080. You should see log messages indicating successful connection to MongoDB and Tomcat initialization.

Step 4: Access the Application

Open your web browser and navigate to:

```
http://localhost:8080
```

The frontend loads automatically. You can log in with the default admin credentials or register a new customer account.

Step 5: (Optional) Set Up CLI

For command-line access:

```
cd cli
```

```
pip install requests rich
```

```
python3 orios.py
```

The interactive menu will guide you through authentication and operations.

Default Credentials

After first startup, an admin account is pre-seeded in the database:

- Email: admin@orios.com
- Password: admin123

It is strongly recommended to change the default admin password in production environments.

! Troubleshooting Tip: If you encounter 'Connection refused' errors, verify MongoDB is running (`ps aux | grep mongod` on Linux/Mac or check Services on Windows). If port 8080 is in use, modify `server.port` in `backend/src/main/resources/application.properties`.

Summary

Orios Bank demonstrates a complete, production-oriented banking application architecture. By separating concerns into distinct layers — Controllers for HTTP handling, Services for business logic, Repositories for data access, and Models for entity definition — the codebase remains organized and testable.

The choice of vanilla JavaScript over a framework like React or Angular keeps the frontend lightweight and dependency-free, while still delivering a modern, animated user experience. The Python CLI extends accessibility to terminal users and automation scripts.

Security is implemented at every layer: BCrypt password hashing, JWT token-based authentication, role-based endpoint protection, and input validation. The MongoDB document store provides the flexibility needed for varied financial entities while maintaining performance.

Quick Reference

Component	Technology	Purpose
Backend API	Java 17 + Spring Boot 3.2	REST API & business logic
Database	MongoDB (localhost:27017)	Document data persistence
Frontend	HTML5 + CSS3 + Vanilla JS	Browser-based user interface
CLI	Python 3.x + requests + rich	Terminal-based API client
Security	JWT + Spring Security + BCrypt	Authentication & authorization
Build Tool	Apache Maven	Dependency management
Default URL	http://localhost:8080	Application access point